ISA 扩展设计说明（A/B/C 指令方案）

1. 设计前提与基本假设

在本方案中，先明确以下设计前提与实现边界：

1. 现有 ISA 以 `opcode[15]` 区分 A/B 两类，且 `0x80~0x83` 已被现有 B 指令占用。
2. 现有 B 指令的两拍执行模型本质上支持两种情况：
3. 第 1 执行拍取第二字；
4. 第 2 执行拍完成“用第二字执行”或“完成一次寄存器间接访存”。
5. 允许对 `controller / execution_stage_fsm / ALU` 做中等规模修改，但**不新增复杂乘法器、堆栈 RAM、Cache 等硬件**。
6. 对 C 指令，采用“**引入新类别前缀 + 新执行状态**”的方式实现。
7. 假设 `R15` 可被定义为 **link register**，用于子程序返回地址保存。

基于以上约束，本文选择新增 5 条指令，分别覆盖： - A 类：单拍 ALU 一元操作； - B 类：双字立即数运算； - C 类：多拍控制流复杂操作。

---

2. 新增指令总览

指令名	类型	指令格式	语义	单/双字	是否访存	是否改动 ALU	是否改动 timer/controller	实现复杂度
NOT	A	`NOT dr`	`dr <- ~dr`	单字	否	是	是	低
ASR	A	`ASR dr`	算术右移一位，符号位保持	单字	否	是	是	低
ADDI	B	`ADDI dr, imm16`	`dr <- dr + imm16`	双字	否	否	是	低
ANDI	B	`ANDI dr, imm16`	`dr <- dr & imm16`	双字	否	否	是	低
CALLA	C	`CALLA addr16`	`R15 <- PC; PC <- addr16`	双字	取第二字	否	是，需新增 C 状态	中

---

3. 指令选择理由

3.1 A 组：NOT 和 ASR

A 组选择 `NOT` 和 `ASR` 的理由如下：

- **教学意义强**：
- `NOT` 补齐基础逻辑一元操作；

- `ASR` 补齐“逻辑右移 vs 算术右移”的组成原理知识点。
- **都能在单执行拍完成**，符合 A 组边界。
- **验证简单**：
- `NOT` 只需检查按位取反结果；
- `ASR` 只需检查符号扩展与最低位移出行为。
- **对原结构侵入小**：
- 只需 ALU 新增两个功能编码；
- 控制器补两条 opcode 即可。

3.2 B 组：ADDI 和 ANDI

B 组选择 `ADDI` 和 `ANDI` 的理由如下：

- 它们都是非常典型的“**双字立即数 ALU 指令**”，与现有 `MVRD`、`JMPA` 同一风格。
- 可以**复用现有 B 类**“先取第二字，再执行”的路径，不需要额外数据访存。
- **易于测试**：
- 一个算术立即数；
- 一个逻辑立即数；
- 可以很好覆盖立即数通路。
- **成体系**：
- 现有已有寄存器-寄存器 `ADD/AND`；
- 现在补齐寄存器-立即数版本，教学上自然且完整。

3.3 C 组：CALLA

C 组选择 `CALLA` 的理由如下：

- **教学意义最好**：
- 引入“子程序调用、返回地址保存、控制流改变、link register”等核心概念。
- **复杂度可控**：
- 不需要栈硬件；
- 不需要乘法器；
- 只需将 `R15` 约定为链接寄存器。
- **易于验证**：
- 检查是否正确读取第二字地址；
- 检查 `R15` 是否保存返回地址；
- 检查 `PC` 是否跳转到目标地址。
- 能清晰形成 A/B/C 三类分层：
- **A**：单拍 ALU；
- **B**：双拍双字；
- **C**：多拍控制流复杂操作。

---

4. 各指令详细规格说明

4.1 A1. NOT

汇编格式

```
NOT dr
```

功能语义

```
dr <- ~dr
```

操作数说明

- `dr`：目标寄存器，同时也是源寄存器。

指令编码建议

- `opcode = 8'h0E`
- 16 位编码格式：

```
00001110 dddd xxxx
```

单字还是双字

- 单字

标志位

建议如下：

- `Z`：按结果是否为 0 更新
- `S`：按结果 `bit15` 更新
- `C`：清 0
- `V`：清 0

执行流程

取指两拍后：

- 执行拍：ALU 读取 `dr`，执行按位取反，结果写回 `dr`，并更新 FLAG。

对现有数据通路的依赖

- 复用 register file；
- 复用单拍 ALU 写回路径；
- 需要 ALU 新增 `NOT` 功能码。

对 controller 的影响

- A 组 opcode 表增加 `0x0E`；
- `writeback_select = WRITEBACK_REG`
- `alu_in_sel = ALU_IN_DR`
- `sst = SST_WRITE`

对 timer / 状态机 的影响

- 无新增状态，仍走 A 组单执行拍。

4.2 A2. ASR

汇编格式

```
ASR dr
```

功能语义

算术右移一位：

```
dr <- {dr[15], dr[15:1]}
```

操作数说明

- `dr`：目标寄存器，同时也是源寄存器。

指令编码建议

- `opcode = 8'h0F`
- 16 位编码格式：

```
00001111 dddd xxxx
```

单字还是双字

- 单字

标志位

建议如下：

- `C`：移出前的 `dr[0]`
- `Z`：结果是否为 0
- `S`：结果 `bit15`
- `V`：清 0

执行流程

取指两拍后：

- 执行拍：ALU 读取 `dr`，执行算术右移，结果写回 `dr`，并更新 FLAG。

对现有数据通路的依赖

- 复用单拍 ALU；
- 需要 ALU 新增 `ASR` 功能码。

对 controller 的影响

- A 组 opcode 表增加 `0x0F`；
- `writeback_select = WRITEBACK_REG`
- `alu_in_sel = ALU_IN_DR`
- `sst = SST_WRITE`

对 timer / 状态机 的影响

- 无新增状态。

4.3 B1. ADDI

汇编格式

```
ADDI dr, imm16
```

功能语义

```
dr <- dr + imm16
```

操作数说明

- `dr`：目标寄存器，同时也是参与运算的原值；
- `imm16`：由第二字给出的 16 位立即数。

指令编码建议

- `opcode = 8'h84`
- 第 1 字：

```
10000100 dddd xxxx
```

- 第 2 字：

```
imm16
```

单字还是双字

- 双字

标志位

与现有 `ADD` 保持一致：

- `C / Z / V / S` 全部按 ALU 加法结果更新。

执行流程

取指两拍后：

- 执行拍 1：`AR <- PC, PC <- PC + 1`，准备读取第二字立即数；
- 执行拍 2：`dr <- dr + MEM[AR]`。

对现有数据通路的依赖

- 完全复用现有 B 组 `MVRD/JMPA` 的第二字读取通路；
- ALU 直接复用 `ADD`。

对 controller 的影响

- B 组 opcode 表增加 `0x84`；
- 第 1 执行拍控制同“取第二字”；
- 第 2 执行拍：
- `alu_in_sel = ALU_IN_MEM`
- `alu_func = ALU_ADD`
- 写回寄存器

对 timer / 状态机 的影响

- 不新增状态，走 B 组两拍执行。

4.4 B2. ANDI

汇编格式

```
ANDI dr, imm16
```

功能语义

```
dr <- dr & imm16
```

操作数说明

- `dr`：目标寄存器；
- `imm16`：第二字立即数。

指令编码建议

- `opcode = 8'h85`
- 第 1 字：

```
10000101 dddd xxxx
```

- 第 2 字：

```
imm16
```

单字还是双字

- 双字

标志位

建议与现有 `AND` 一致：

- `Z`：结果是否为 0
- `S`：结果 `bit15`
- `C`：清 0
- `V`：清 0

执行流程

取指两拍后：

- 执行拍 1：`AR <- PC, PC <- PC + 1`
- 执行拍 2：`dr <- dr & MEM[AR]`

对现有数据通路的依赖

- 复用现有 B 组第二字读取通路；
- 复用现有 ALU `AND`。

对 controller 的影响

- B 组 opcode 表增加 `0x85`；
- 第 2 执行拍选择：
- `ALU_IN_MEM`
- `ALU_AND`

对 timer / 状态机 的影响

- 不新增状态。

4.5 C1. CALLA

汇编格式

```
CALLA addr16
```

功能语义

绝对调用：

```
R15 <- PC
PC  <- addr16
```

其中，`PC` 保存的是“调用指令之后下一条指令地址”，即返回地址。

操作数说明

- `addr16`：第二字给出的 16 位绝对目标地址；
- `R15`：隐含 `link register`。

指令编码建议

- `opcode = 8'hF0`
- 第 1 字：

```
11110000 0000 0000
```

- 第 2 字：

```
addr16
```

单字还是双字

- 双字

标志位

- 不修改 FLAG，`sst = SST_HOLD`

执行流程

取指两拍后：

- 执行拍 1：`AR <- PC, PC <- PC + 1`，读取第二字目标地址；
- 执行拍 2：`R15 <- PC`，保存返回地址；
- 执行拍 3：`PC <- MEM[AR]`，跳转到目标地址。

对现有数据通路的依赖

- 第 1 拍完全复用现有 B 组“取第二字”；
- 第 2 拍用 ALU 的 `PC -> 寄存器写回` 路径，把 `PC` 写到 `R15`；
- 第 3 拍用现有 `MEM[AR] -> PC` 路径完成跳转；
- 不需要新寄存器；
- 不需要栈。

对 controller 的影响

- 新增 C 组 opcode `0xF0`；
- 第 2 拍需要强制 `dest_reg = 4'hF`；
- 第 2 拍：
- `writeback_select = WRITEBACK_REG`
- `alu_in_sel = ALU_IN_PC`
- 第 3 拍：
- `writeback_select = WRITEBACK_PC`
- `alu_in_sel = ALU_IN_MEM`
- `sst = SST_HOLD`

对 timer / 状态机 的影响

需要新增 C 类状态分支。建议如下：

在取指第 2 拍后判断：

- `opcode[15] == 0` → A
- `opcode[15:8] == 8'hF0` → C
- 否则 → B

新增两个状态，例如：

- `STAGE_C_SAVE_RETURN`
- `STAGE_C_LOAD_TARGET`

---

5. 规则文件 / 汇编语法建议

可按现有风格扩展为：

```
NOT dr        "00001110[u4]xxxx",dr
ASR dr        "00001111[u4]xxxx",dr
ADDI dr,data  "10000100[u4]xxxx[u16]",dr,data
ANDI dr,data  "10000101[u4]xxxx[u16]",dr,data
CALLA addr    "1111000000000000[u16]",addr
```

---

6. A/B/C 分类依据说明

6.1 为什么 NOT、ASR 属于 A

- **执行拍数**：取指后 1 拍完成；
- **是否需要访存**：不需要；
- **是否需要第二字**：不需要；
- **是否需要暂存**：不需要；
- **是否需要新增状态**：不需要；
- **是否可在单拍 ALU 路径完成**：可以，都是单寄存器一元运算。

6.2 为什么 ADDI、ANDI 属于 B

- **执行拍数**：取指后 2 拍完成；
- **是否需要访存**：不需要额外数据访存，但需要读取第二字立即数；
- **是否需要第二字**：需要；
- **是否需要暂存**：不需要额外暂存器；
- **是否需要新增状态**：不需要，复用现有 B 类两拍；
- **是否可在单拍 ALU 路径完成**：最终运算本身能单拍完成，但必须先取第二字，因此归入 B。

6.3 为什么 CALLA 属于 C

- **执行拍数**：取指后 3 拍完成；
- **是否需要访存**：需要读取第二字目标地址；
- **是否需要第二字**：需要；
- **是否需要暂存**：不需要新硬件暂存，但需要分两步完成“保存返回地址”和“更新 PC”；
- **是否需要新增状态**：需要，至少新增 1~2 个 C 类执行状态；
- **是否可在单拍 ALU 路径完成**：不能。

其本质原因在于，该指令包含两个顺序写回目标：

1. 先写 `R15`；
2. 再写 `PC`。

在现有单写回结构下，无法在单拍内同时完成，因此必须定义为 C 类。

---

7. 方案总结

本 ISA 扩展方案的核心特点是：

- **A 类**补齐基础单目逻辑与移位能力；
- **B 类**补齐寄存器-立即数运算能力；
- **C 类**引入最基本的子程序调用能力；
- 在保持原有总体数据通路框架不变的前提下，以**较低硬件代价**实现较完整的教学型 ISA 扩展。

从工程实现角度看，该方案有以下优点：

- 改动范围集中在 `controller / execution_stage_fsm / ALU`；
- 不引入复杂新硬件模块；
- 与现有 A/B 指令执行框架兼容性较好；
- 易于形成后续实验内容，包括：

- 控制信号表设计；
- 状态机扩展；
- 汇编器规则扩展；
- 功能仿真测试程序。

---

8. 可继续展开的后续工作

在此文档基础上，下一步可以继续细化为以下实现材料：

1. `controller` 的节拍级真值表；
2. `execution_stage_fsm` 的新增状态转移图；
3. `ALU` 需要增加的功能码与组合逻辑修改建议；
4. 对应的汇编测试程序与验证用例；
5. `CALLA` 配套的 `RET` 候选设计（若后续需要完整子程序机制）。